

I. LEARNING OBJECTIVES

At the end of this activity, students should be able to:

1. Integrate EJS into an existing Express.js application.
2. Configure Express.js to use EJS as its view engine.
3. Create and organize EJS view files.
4. Pass server-side data from Express routes to EJS views.
5. Display dynamic data using EJS syntax.
6. Use EJS JavaScript logic such as `if` statements and `forEach()` loops.
7. Create a reusable master layout.
8. Create reusable partial components such as headers and footers.
9. Integrate server-side rendered pages with the existing Express.js project.

II. IMPORTANT INSTRUCTION

DO NOT CREATE A NEW EXPRESS PROJECT.

Continue using the **same Express.js project** that you used in:

- HTTP Methods
- Middleware Integration

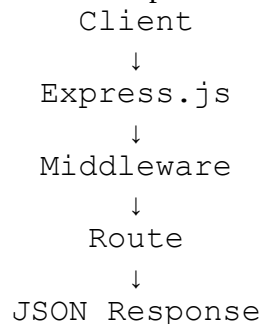
Your existing project should already contain your API routes and middleware.

You will now add a **server-side rendered interface using EJS**.

You are **enhancing your existing project**, not replacing it.

III. WHAT ARE WE BUILDING?

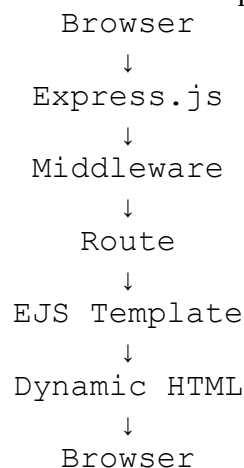
Previously, your application focused mainly on API responses:



Example:

```
{
  "name": "Juan Dela Cruz",
  "course": "BSIT",
  "year": 1
}
```

In this activity, we will allow Express.js to generate an HTML page dynamically:



A templating engine allows developers to generate dynamic HTML pages using server-side data. EJS is the templating engine used in this module.

IV. ACTIVITY 1 – CHECK YOUR EXISTING PROJECT

Open your previous Express.js project.

Your project should contain something similar to:

```
your-project/
├── middleware/
│   ├── logger.js
│   ├── auth.js
│   ├── role.js
│   └── errorHandler.js
├── app.js
├── package.json
└── package-lock.json
```

Checkpoint

Run your application.

Test one of your existing routes.

Example:

GET `http://localhost:3000/students`

Make sure your existing application works before proceeding.

V. ACTIVITY 2 – INSTALL EJS

We will now add EJS to the **existing project**.

Open the terminal inside your project folder.

Run:

```
npm install ejs
```

The module identifies `npm install ejs` as the command for installing the EJS dependency.

Guide Question 1

Why do we need to install EJS before we can use it?

Answer:

VI. ACTIVITY 3 – CONFIGURE EJS

Open your existing `app.js`.

After creating your Express application:

```
const express = require('express');
```

```
const app = express();
```

add:

```
app.set('view engine', 'ejs');
```

Your code should now contain:

```
const express = require('express');
```

```
const app = express();
```

```
app.set('view engine', 'ejs');
```

The module specifies `app.set('view engine', 'ejs')` as the Express configuration for establishing EJS as the view engine.

Guide Question 2

What does this line do?

```
app.set('view engine', 'ejs');
```

Answer:

VII. ACTIVITY 4 – CREATE THE VIEWS FOLDER

At the root of your existing project, create:

`views`

Your project should now look like:

`your-project/`

```
├── middleware/
├── views/
├── app.js
├── package.json
└── package-lock.json
```

Express conventionally looks for view/template files in the `views` directory. The module specifically instructs students to establish a `/views` folder for the interface assets.

VIII. ACTIVITY 5 – CREATE YOUR FIRST EJS VIEW

Inside the `views` folder, create:

`index.ejs`

Your structure:

```
views/
├── index.ejs
```

Inside `index.ejs`, write:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>My Express Application</title>
</head>
<body>

  <h1>Welcome to My Express Application</h1>

  <p>This page is rendered using EJS.</p>

</body>
</html>
```

IX. ACTIVITY 6 – CREATE AN EJS ROUTE

Go to your existing `app.js`.

Add a new route:

```
app.get('/', (req, res) => {  
  res.render('index');  
});
```

Important

Notice that we are using:

```
res.render()
```

instead of:

```
res.send()
```

The module identifies `res.render()` as the Express response method used to process a template and return the resulting HTML to the client.

X. ACTIVITY 7 – TEST YOUR FIRST EJS PAGE

Start your server.

Open:

```
http://localhost:3000/
```

You should see:

```
Welcome to My Express Application
```

Guide Question 3

What is the difference between:

```
res.send()
```

and:

```
res.render()
```

Answer:

XI. ACTIVITY 8 – PASS SERVER DATA TO EJS

Now we will make the page dynamic.

Modify your `/` route:

```
app.get('/', (req, res) => {  
  
  const student = {  
    name: 'Juan Dela Cruz',  
    course: 'BSIT',  
    year: 1  
  };  
  
  res.render('index', { student });  
});
```

The object:

```
{  
  name: 'Juan Dela Cruz',  
  course: 'BSIT',  
  year: 1  
}
```

is being passed to the EJS template.

The module demonstrates passing server-side data to a view through the object supplied as the second argument of `res.render()`.

XII. ACTIVITY 9 – DISPLAY THE DATA USING EJS

Open:

views/index.ejs

Add:

```
<h2>Student Information</h2>
```

```
<p>Name: <%= student.name %></p>
```

```
<p>Course: <%= student.course %></p>
```

```
<p>Year Level: <%= student.year %></p>
```

Your page should display:

Student Information

Name: Juan Dela Cruz

Course: BSIT

Year Level: 1

The `<%= %>` expression outputs evaluated data into the page while escaping HTML characters.

XIII. ACTIVITY 10 – UNDERSTAND `<%= %>`

Consider:

```
<p>Name: <%= student.name %></p>
```

Answer the following:

1. What is **student**?

2. What is **name**?

3. What does `<%= %>` do?

XIV. ACTIVITY 11 – USE YOUR EXISTING DATA

Now connect EJS to the data you already used in your previous HTTP Methods project.

If you previously created:

```
const students = [
  {
    id: 1,
    name: 'John Doe',
    course: 'BSIT'
  },
  {
    id: 2,
    name: 'Jane Smith',
    course: 'BSIT'
  }
];
```

you will now display that data through an EJS page.

Add a route:

```
app.get('/students/view', (req, res) => {
  res.render('students', { students });
});
```

XV. ACTIVITY 12 – CREATE `students.ejs`

Inside the views folder:

views/

├─ index.ejs

└─ students.ejs

Create:

students.ejs

Add:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Students</title>
</head>
<body>

  <h1>Student List</h1>

</body>
</html>
```

XVI. ACTIVITY 13 – DISPLAY MULTIPLE STUDENTS

Inside students.ejs, add:

```
<ul>

  <% students.forEach(student => { %>

    <li>
      <%= student.name %> -
      <%= student.course %>
    </li>

    <% }); %>

</ul>
```

The EJS `<% %>` tag is used to execute JavaScript logic without directly printing the JavaScript statement to the page. It is appropriate for loops and conditions.

XVII. ACTIVITY 14 – UNDERSTAND THE LOOP

Study:

```
<% students.forEach(student => { %>

  <li>
    <%= student.name %>
  </li>

  <% }); %>
```

Answer:

1. What does `students.forEach()` do?

2. Why do we use `<% %>` around the `forEach()`?

3. Why do we use `<%= student.name %>`?

4. What will happen if there are 10 students in the array?

XVIII. ACTIVITY 15 – ADD A CONDITIONAL

Inside `students.ejs`, add:

```
<% if (students.length > 0) { %>

    <p>There are students registered in the system.</p>

<% } else { %>

    <p>No students found.</p>

<% } %>
```

This demonstrates how EJS can execute JavaScript conditions inside a template.

XIX. ACTIVITY 16 – CREATE A VIEWS/PARTIALS FOLDER

Now we will organize reusable interface components.

Inside `views`, create:

`partials`

Your structure becomes:

```
views/
├── index.ejs
├── students.ejs
└── partials/
```

XX. ACTIVITY 17 – CREATE A HEADER PARTIAL

Inside:

`views/partials/`

create:

`header.ejs`

Add:

```
<header>

    <h1>My Express Application</h1>

    <nav>
        <a href="/">Home</a>
        |
        <a href="/students/view">Students</a>
    </nav>

    <hr>

</header>
```

Partials are reusable view fragments used to avoid repeating common markup such as navigation and headers.

XXI. ACTIVITY 18 – CREATE A FOOTER PARTIAL

Inside:

`views/partials/`

create:

`footer.ejs`

Add:

```
<hr>
```

```
<footer>
  <p>&copy; 2026 My Express Application</p>
</footer>
```

Your folder should now be:

```
views/
├── index.ejs
├── students.ejs
└── partials/
    ├── header.ejs
    └── footer.ejs
```

XXII. ACTIVITY 19 – INCLUDE THE HEADER

Open:

views/index.ejs

At the beginning of the <body>, add:

```
<%- include('partials/header') %>
```

For example:

```
<body>

  <%- include('partials/header') %>

  <h1>Welcome to My Express Application</h1>

</body>
```

The module specifies the include expression:

```
<%- include('partials/filename') %>
```

for importing reusable partial components.

XXIII. ACTIVITY 20 – INCLUDE THE FOOTER

Before the closing </body> tag, add:

```
<%- include('partials/footer') %>
```

Your page should now have:

```
Header
  ↓
Page Content
  ↓
Footer
```

XXIV. ACTIVITY 21 – APPLY PARTIALS TO students.ejs

Now open:

views/students.ejs

Add:

```
<%- include('partials/header') %>
```

at the beginning of the body.

Then add:

```
<%- include('partials/footer') %>
```

before the closing body tag.

Test:

http://localhost:3000/

and:

http://localhost:3000/students/view

Both pages should have the same header and footer.

XXV. ACTIVITY 22 – CREATE A MASTER LAYOUT

We will now create a reusable master frame.

Inside:

views/

create:

layout.ejs

Your structure:

views/

```
├── layout.ejs
├── index.ejs
├── students.ejs
└── partials/
    ├── header.ejs
    └── footer.ejs
```

Inside layout.ejs:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">

  <title>My Express Application</title>
</head>

<body>

  <%- include('partials/header') %>

  <main>
    <%- body %>
  </main>

  <%- include('partials/footer') %>

</body>
</html>
```

The module describes layout.ejs as a master wrapper that provides a common structural frame for individual views.

XXVI. ACTIVITY 23 – UNDERSTAND THE LAYOUT

Study:

```
<main>
```

```
  <%- body %>
```

```
</main>
```

Question:

What is the purpose of:

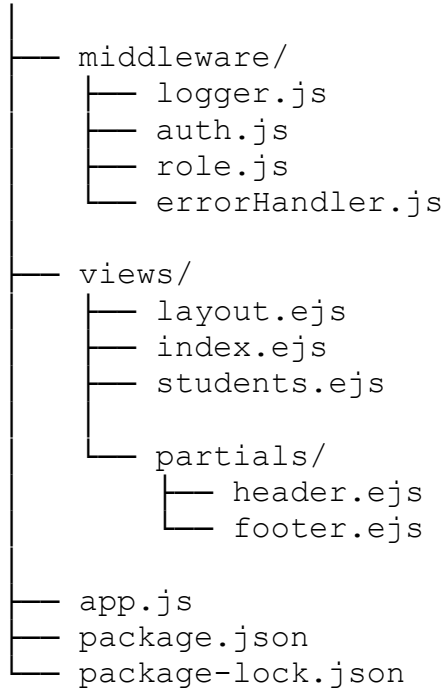
```
<%- body %>
```

Answer:

XXVII. ACTIVITY 24 – FINAL PROJECT INTEGRATION

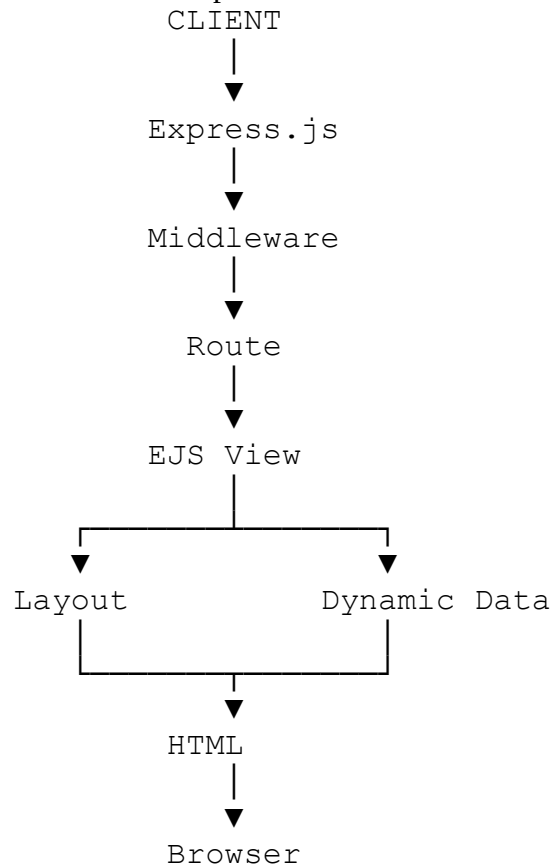
Your existing project should now have:

your-project/



XXVIII. FINAL APPLICATION FLOW

Your project now combines the previous lessons:



XXIX. FINAL CHALLENGE

Now modify your existing project.

Create a new page:

/dashboard

The dashboard must display:

Welcome, [Student Name]

Course: BSIT

Year Level: 1

Total Students: [number]

Use:

- An Express route
- `res.render()`
- An EJS view
- Server-side data
- `<%= %>`
- At least one `<% %>` statement
- Header partial
- Footer partial
- Your existing middleware

XXX. FINAL TESTING CHECKLIST

Requirement	Completed
Existing HTTP Methods still work	☐
Existing middleware still works	☐
EJS installed	☐
EJS configured as view engine	☐
views folder created	☐
index.ejs created	☐
<code>res.render()</code> used	☐
Server data passed to EJS	☐
<code><%= %></code> used	☐
<code><% %></code> used	☐
<code>forEach()</code> used	☐
Conditional statement used	☐
partials folder created	☐
Header partial created	☐
Footer partial created	☐
Master layout created	☐
Dashboard created	☐

XXXI. DEBUGGING CHALLENGE

A student created this route:

```
app.get('/students/view', (req, res) => {
  res.render('students');
});
```

But inside `students.ejs` they have:

```
<h1>Student List</h1>
```

```
<% students.forEach(student => { %>
```

```
<p><%= student.name %></p>
```

```
<% }); %>
```

The application produces an error.

Question:

What is missing from the `res.render()` statement?

Correct the code:

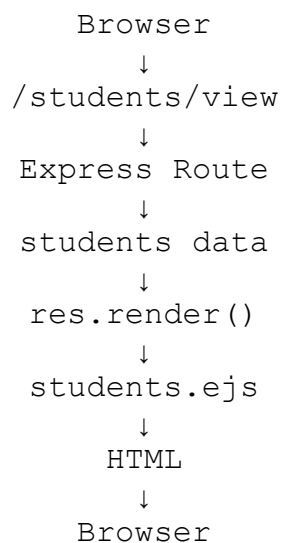
```
app.get('/students/view', (req, res) => {  
  
    res.render('students', _____);  
  
});
```

XXXII. CONCEPTUAL CHALLENGE

Consider:

```
app.get('/students/view', (req, res) => {  
    res.render('students', { students });  
});
```

Explain the complete process:



Your explanation:

XXXIII. REFLECTION

Answer in 5–7 sentences:

How does integrating EJS change your existing Express.js project from a simple API application into a server-rendered web application?

XXXIV. SUBMISSION REQUIREMENTS

Submit your **updated existing Express.js project**.

You are **not required to create a new project**.

Your project must demonstrate:

Previous Lessons

- ☐ HTTP Methods are still functional.
- ☐ Middleware is still functional.
- ☐ Middleware remains organized in the `middleware` folder.

Module 4

- ☐ EJS is installed.
- ☐ EJS is configured as the view engine.
- ☐ `views` folder is created.
- ☐ Dynamic data is passed from Express to EJS.
- ☐ `<%= %>` is used.
- ☐ `<% %>` is used.
- ☐ A `forEach()` loop is implemented.
- ☐ A conditional statement is implemented.
- ☐ `partials` folder is created.
- ☐ Header partial is implemented.
- ☐ Footer partial is implemented.
- ☐ Master layout is implemented.
- ☐ Dashboard page is implemented.

XXXV. KEY TAKEAWAY

You have now progressed from:

HTTP Methods

↓

Middleware

↓

EJS

↓

Dynamic Data

↓

Layouts

↓

Partials

Your Express.js application is no longer limited to returning JSON responses.

It can now:

receive a request → process it through middleware → retrieve data → render an EJS template → generate HTML → send the dynamic page to the browser.

This reflects the module's focus on bridging static content and responsive, data-driven web applications through server-side rendering.